

AISM-Konformität

Konformitätsstufen, Testkatalog, Testdaten und Berichtsformat für AISM-Implementierungen

Dokumentstatus	Entwurf 0.2 (Testsuite 0.2.0, Nachtrag 05.10.2026: K2-13 automatisiert, K2-19 bis K2-21, K3-08 automatisiert, K3-09, K3-10)
Datum	05.10.2026
Bezug	../AISM-Spezifikation.md §9 (M-xx, S-xx), ../policy/AISM-Policy-Format.md , Testsuite tests/
Normative Sprache	MUSS / SOLLTE gemäß RFC 2119

AISM-Konformität ist eine **technische Selbstprüfung** gegen diese Spezifikation. Sie ist keine Zertifizierung und belegt keine Einhaltung rechtlicher Anforderungen (z. B. Datenschutzrecht). Die Bezeichnung „Sovereign“ beschreibt nur die technischen Eigenschaften der Stufe K3.

1. Konformitätsstufen

Die Stufen heißen **K1-K3**, damit sie nicht mit den AISM-Stufen S1-S7 oder den Komponentengruppen L1-L7 verwechselt werden. Jede Stufe schließt die darunterliegenden ein.

Stufe	Name	Kurzbeschreibung	Erforderliche Kriterien (Spez. §9)
K1	Basic	Gateway erzwungen, OpenAI-kompatible API inkl. SSE, Health, Authentifizierung, kein Bypass	M-01, M-06, M-13 sowie §6.2 Schritt 1 (AuthN)
K2	Governed	K1 + PII-Maskierung, Tool-Allowlist und Schemaprüfung, Audit ohne Klartext-PII, RAG nur nach Maskierung, gültige Policy geladen, Cloud standardmäßig aus	zusätzlich M-02, M-04, M-05, M-07, M-08, M-09, M-11, M-12, M-15
K3	Sovereign/Auditable	K2 + signierte Policy-Versionen, Trace-IDs Ende-zu-Ende, Cloud-Fallback mit Nachweis, reproduzierbares Deployment (Digests), manipulationserkennbares Audit	zusätzlich S-02, S-07, S-10, S-11, S-13, die in K3 als Pflicht gelten

Eine Stufe gilt als **erreicht**, wenn alle MUSS-Tests dieser und aller darunterliegenden Stufen **bestanden** sind. Übersprungene MUSS-Tests (fehlende Voraussetzung) gelten als **nicht erreicht**. Fehlgeschlagene SOLLTE-Tests verhindern die Stufe nicht, werden aber im Bericht ausgewiesen.

2. Prüfaufbau

```
flowchart LR
    R["Test-Runner (pytest)<br/>im frontend-Netz"] -->|" /v1/*, /health, /aism/v1/policy"| S2["S2 Gateway"]
    S2 --> S3["S3 Orchestrator"]
    S3 -->|"Inferenz, Embeddings, Webhooks"| M["Capture-Mock<br/>(statt S6, S4-Embedding, n8n)"]
    R -->|"GET /_captured"| M
    R -->|"liest"| A["Audit-Log"]
    R -->|"liest"| F["docker-compose.yml<br/>policy.yaml"]
```

- **Live-Tests** sprechen das Gateway an wie ein Client.
- **Capture-Tests** prüfen, was hinter S2/S3 ankommt. Dazu ersetzt die Compose-Erweiterung [tests/compose.conformance.yml](#) die Upstreams des Orchestrators durch den Capture-Mock [tests/mock_upstream.py](#). Qdrant und SearXNG bleiben real.
- **Audit-Tests** lesen das Audit-Log (JSONL).
- **Statische Tests** prüfen Compose-Datei und Policy.
- **Manuelle Prüfungen** erfordern Fehlerinjektion oder kryptografische Verifikation außerhalb der Suite.

Der Bypass-Test (AISM-K1-05) ist nur aussagekräftig, wenn der Runner im **Client-Netz** läuft (z. B. Container im Netz `aism_frontend`), weil Backend-Dienstnamen dort nicht auflösbar bzw. erreichbar sein dürfen.

3. Testkatalog

Spalte **Art**: `live`, `capture`, `audit`, `static`, `manuell`. Spalte **Status**: `impl.` = in [tests/](#) implementiert; `manuell` = Prüfanweisung, kein automatisierter Test.

3.1 K1 Basic

ID	Anf.	Spez.	Art	Verfahren	Erwartetes Ergebnis	Status
AISM-K1-01	MUSS	§6.2 (/health)	live	GET /health	200	impl.
AISM-K1-02	MUSS	M-06	live	GET /v1/models	object: "list", nicht-leeres data[] mit id	impl.
AISM-K1-03	MUSS	M-06	live	POST /v1/chat/completions (nicht streamend)	object: "chat.completion", choices[0].message.role = "assistant", gültiger finish_reason	impl.
AISM-K1-04	MUSS	M-06, §6.6	live	wie K1-03 mit stream: true	Content-Type: text/event-stream; jede data: -Zeile ist chat.completion.chunk-JSON; mind. ein finish_reason; Abschluss data: [DONE]; nichts danach	impl.
AISM-K1-05	MUSS	M-01, S-06	live	HTTP-Abruf aller URLs aus AISM_DIRECT_URLS (S3, S4, S6) aus dem Client-Netz	keine HTTP-Antwort (Verbindungsfehler/Timeout)	impl.
AISM-K1-06	MUSS	§6.2 Schritt 1, PEP-1	live	Chat-Request ohne Zugangsdaten	401	impl.
AISM-K1-07	SOLLTE	§6 Fehlerformat	live	Chat-Request ohne messages	4xx mit OpenAI-Fehlerobjekt {"error": {"message": ...}}	impl.
AISM-K1-08	MUSS	M-01, §6.1	static	Compose: Umgebung von open-webui	OPENAI_API_BASE_URL zeigt auf governance-proxy; kein OLLAMA_BASE_URL; ENABLE_OLLAMA_API=false; ENABLE_DIRECT_CONNECTIONS=false	impl.
AISM-K1-09	MUSS	M-01, S-06	static	Compose: llama-server, llama-embed	keine ports:, kein network_mode: host	impl.
AISM-K1-10	MUSS	M-13	static	Compose: alle Dienste	kein privileged: true	impl.

3.2 K2 Governed

ID	Anf.	Spez.	Art	Verfahren	Erwartetes Ergebnis	Status
AISM-K2-01	MUSS	M-15	live	GET /aism/v1/policy	200 mit name, version, digest (sha256:...); keine Regelinhalte	impl.
AISM-K2-02	MUSS	M-15	static	Policy gegen policy.schema.json validieren	keine Schemafehler	impl.
AISM-K2-03	MUSS	M-02, PEP-2	capture	Prompt mit synthetischer PII (§4) senden; Inferenz-Request am Mock prüfen	keine Klartextwerte; Platzhalter <PERSON_, <EMAIL_, <SECRET_ vorhanden	impl.
AISM-K2-04	MUSS	M-02, M-09, PEP-5	capture	Prompt mit PII und RAG-Bezug; Embedding-Requests am Mock prüfen	keine Klartextwerte in /v1/embeddings; übersprungen, wenn kein Embedding-Request beobachtet wird	impl.
AISM-K2-05	MUSS	M-07, M-08, PEP-7	capture	Mock schlägt Tool delete_ticket (nicht in der Allowlist) vor	kein Webhook-Aufruf; Folge-Request an S6 enthält role: "tool" mit passender tool_call_id und Fehlerinhalt	impl.
AISM-K2-06	MUSS	M-08, PEP-7	capture	Mock schlägt ticket_status_lookup mit schemawidrigem Argument vor	kein Webhook-Aufruf; role: "tool"-Fehlermeldung	impl.
AISM-K2-07	SOLLTE	Policy-Format §4.7	capture	tools-Array der Requests an S6 prüfen	nur in der Policy definierte Tools	impl.
AISM-K2-08	MUSS	M-05, AUD	audit	Audit-Log nach synthetischen PII-Werten durchsuchen	kein Treffer	impl.
AISM-K2-09	MUSS	M-15, AUD	audit	Felder policy.version, policy.digest in jedem Eintrag	vorhanden	impl.
AISM-K2-10	MUSS	M-04, PEP-4	live	bei cloudEnabled: false ein Cloud-Modell aus der Policy anfordern	400 / 403 / 404 mit Fehlerobjekt, oder Antwort nicht vom Cloud-Modell	impl.
AISM-K2-11	MUSS	M-02, §6.1	static	Compose: Open-WebUI-Variablen für RAG/Web-Suche	Werte gemäß Spez. §6.1; keine UI-eigene RAG-Konfiguration (VECTOR_DB, RAG_EMBEDDING_ENGINE)	impl.
AISM-K2-12	MUSS	M-15	static	Compose: Volumes von governance-proxy	read-only-Mount, der POLICY_PATH enthält (Datei ...:/etc/aism/policy.yaml:ro oder Verzeichnis ...:/etc/aism/policy:ro)	impl.

ID	Anf.	Spez.	Art	Verfahren	Erwartetes Ergebnis	Status
AISM-K2-13	MUSS	M-11, PEP-6	capture	Mock schlägt immer <code>web_search</code> vor (Marker <code>AISM-TEST:WEB_SEARCH</code>); Anfrage, die die Policy nicht suchen lässt (<code>webSearch.enabled: false</code> oder Datenklasse über <code>maxDataClassRank</code> , hier wegen E-Mail-Adresse)	<code>web_search</code> nicht angeboten; keine Anfrage am SearXNG-Mock (<code>/search</code>)	impl.
AISM-K2-14	MUSS	M-12	manuell	PII-Detektor (z. B. NER-Dienst) stoppen; Anfrage mit Cloud-Route stellen	Ablehnung; kein Egress; Audit <code>request.denied</code>	manuell
AISM-K2-15	SOLLTE	S-12	static	Compose: Netz <code>backend</code> und Zuordnung von S3, S4, S6	<code>internal: true</code> ; diese Dienste nur in <code>backend</code>	impl.
AISM-K2-16	MUSS	M-07, §6.3	capture	Positivkontrolle: Mock schlägt <code>ticket_status_lookup</code> mit gültigen Argumenten vor	genau ein Webhook-Aufruf mit unveränderten Argumenten; Folge-Request mit <code>assistant(tool_calls)</code> und <code>role: "tool"</code>	impl.
AISM-K2-17	SOLLTE	PEP-10	manuell	Antwort mit Platzhaltern für Rolle mit und ohne <code>demaskFor</code> abrufen	Demaskierung nur für berechtigte Rolle; SECRET nie	manuell
AISM-K2-19	SOLLTE	PEP-6, PEP-8, S-05	capture	Erlaubte Suche (ohne PII in der Nutzerfrage, Ergebnis des Mocks enthält eine E-Mail-Adresse)	Suchanfrage am Mock ohne Klartext-PII und ohne Platzhalter; Ergebnis im Folge-Request an S6 maskiert und als <code>untrusted_web_results</code> gekennzeichnet; Audit <code>egress.websearch</code> ohne Anfragetext	impl.
AISM-K2-20	SOLLTE	S-03, §6.3.1	capture	Mock schlägt schreibendes Tool <code>ticket_add_comment</code> vor (Marker <code>AISM-TEST:WRITE_TOOL</code>)	kein Webhook-Aufruf vor Bestätigung; <code>pending_confirmation</code> ; Liste/Bestätigen nur für denselben Benutzer (fremd/anonym <code>401/404</code>); nach <code>approve</code> genau ein Webhook-Aufruf; erneutes <code>approve 409</code> ; <code>reject</code> führt nicht aus	impl.
AISM-K2-21	SOLLTE	S-01, PEP-1	live	<code>Authorization: Bearer <IdP-Token></code> (<code>AISM_OIDC_TOKEN</code>) und Negativ-Tokens (<code>AISM_OIDC_NEGATIVE_TOKENS</code> : abgelaufen, falsche <code>aud</code> , fremder Schlüssel, falscher <code>iss</code>) sowie Kopie mit verfälschter Signatur	gültiges Token <code>200</code> ; alle anderen <code>401</code>	impl.
AISM-K2-18	SOLLTE	PEP-10, §6.6	capture	Mock-Marker <code>AISM-TEST:ECHO</code> : Mock streamt den maskierten Text in 3-Zeichen-Chunks zurück	keine PII an S6; im Client-Stream jeder Wert entweder im Klartext (demaskiert) oder als vollständiger Platzhalter, nie als Fragment	impl.

3.3 K3 Sovereign/Auditable

ID	Anf. (in K3)	Spez.	Art	Verfahren	Erwartetes Ergebnis	Status
AISM-K3-01	MUSS	S-11	static	<code>metadata.signature</code> und <code>metadata.revision</code> der Policy; mit <code>AISM_ALLOWED_SIGNERS</code> zusätzlich Signaturprüfung (SSHSIG) der Datei	vorhanden; Signatur gültig	impl.
AISM-K3-02	MUSS	S-10	capture	Request mit W3C- <code>traceparent</code> ; Header am Mock prüfen	gleiche Trace-ID im Inferenz-Request	impl.
AISM-K3-03	MUSS	S-10, AUD	audit	<code>trace_id</code> in request-bezogenen Audit-Einträgen	32 Hex-Zeichen	impl.
AISM-K3-04	MUSS	S-13, M-04	audit	Lauf mit Test-Policy <code>cloudEnabled: true</code> und Mock-Cloud-Provider; <code>egress.cloud</code> -Einträge prüfen	jeder Eintrag enthält <code>rule_ids</code> , <code>provider</code> , <code>policy</code> , <code>payload_sha256</code>	impl. (Szenario: <code>scenario_cloud_fallback.py</code>)
AISM-K3-05	MUSS	S-07	static	Compose: alle <code>image:</code> ; bei Diensten mit <code>build: alle FROM</code> -Zeilen des Dockerfiles	mit <code>@sha256:-Digest</code> gepinnt (das lokal gebaute Image selbst erst nach Push in eine Registry, README)	impl.

ID	Anf. (in K3)	Spez.	Art	Verfahren	Erwartetes Ergebnis	Status
AISM-K3-06	MUSS	S-07	manuell	Prüfsummen der Modelldateien gegen dokumentierte Werte	übereinstimmend	manuell
AISM-K3-07	MUSS	S-02	audit	Hash-Kette: <code>prev_hash</code> jedes Eintrags = <code>sha256: + SHA-256</code> der vorherigen Zeile (AISM-Format)	lückenlos	impl.
AISM-K3-08	MUSS	S-11	live (Fehlerinjektion, opt-in <code>AISM_POLICY_FAULT_INJECTION=1</code> , Marker <code>inject</code>)	gemountete Policy nacheinander durch unsigned und durch manipulierte Fassung (gültige alte Signatur) ersetzen, dann Original wiederherstellen	Digest von <code>/aism/v1/policy</code> unverändert; <code>policy.load_failed</code> mit <code>kept_active: true</code> im Audit	impl.
AISM-K3-09	MUSS	S-11	live	GET <code>/aism/v1/policy</code>	<code>revision</code> gesetzt; <code>signature.verified: true</code> und <code>signature.required: true</code>	impl.
AISM-K3-10	SOLLTE	M-04, PEP-4, PEP-9, S-13	Szenario	lokales Modell gestoppt, Cloud-Test-Policy, HTTPS-Mock-Provider (<code>compose.cloud-fallback.yml</code>)	ohne PII und mit Rolle <code>it-ops</code> : Antwort vom Cloud-Modell (JSON und Stream), keine X-AISM-* -Header beim Provider, Audit <code>egress.cloud</code> mit <code>reason: local_unavailable</code> ; mit PII bzw. ohne Rolle: 503, nichts beim Provider	impl.

4. Testdaten

Datei: `tests/testdata/synthetic_pii.json`. Alle Werte sind **synthetisch**:

Entität	Werte	Herkunft
PERSON	„Erika Mustermann“, „Max Mustermann“	gängige deutsche Platzhalternamen
EMAIL	<code>erika.mustermann@example.com</code> , <code>max.mustermann@example.org</code>	reservierte Domains nach RFC 2606
SECRET	<code>sk-aismtest...</code> , <code>AKIAIOSFODNN7EXAMPLE</code> , <code>ghp_AISMTEST...</code>	ungültige Beispielwerte; der AWS-Wert ist der Beispielschlüssel aus der AWS-Dokumentation
IBAN (optional)	<code>DE89370400440532013000</code>	verbreitete Beispiel-IBAN; nur bei konfiguriertem IBAN-Detektor

Regeln: Testdaten DÜRFEN keine echten Personen, Adressen oder Zugangsdaten enthalten. Jeder Test verwendet zusätzlich eine zufällige Nonce (`aism-<hex>`), um seine Requests im Capture-Puffer eindeutig zu finden.

Die Tests prüfen nur die in den Testdaten enthaltenen Werte. Das Bestehen von AISM-K2-03 belegt **keine** allgemeine Erkennungsquote. Die Erkennungsqualität ist mit eigenen, repräsentativen Testdaten zu messen (Spez. §10).

5. Ausführung

Siehe `tests/README.md`. Kurzform:

```
cd conformance/tests && pip install -r requirements.txt
python3 -m pytest -m static           # ohne laufenden Stack
python3 -m pytest                     # alle Tests; fehlende Voraussetzungen -> skipped
```

6. Berichtsformat

6.1 JSON-Ergebnis (aism-report.json)

```
{
  "aism_report_version": "1",
  "suite_version": "0.2.0",
  "generated_at": "2026-10-05T01:30:00+02:00",
  "target": {
    "base_url": "http://governance-proxy:8000",
    "compose_file": "/repo/docker-compose.yml",
    "policy_file": "/repo/policy/policy.yaml"
  },
  "summary": {"passed": 24, "failed": 1, "skipped": 5},
  "levels": {
    "K1": {"name": "Basic", "achieved": true, "must_failed": [], "must_not_run": [], "should_failed": []},
    "K2": {"name": "Governed", "achieved": true, "must_failed": [], "must_not_run": [], "should_failed": ["AISM-K2-07"]},
    "K3": {"name": "Sovereign/Auditabile", "achieved": false, "must_failed": ["AISM-K3-05"], "must_not_run": ["AISM-K3-04"], "should_failed": []}
  },
  "achieved_level": "K2",
  "results": [
    {
      "id": "AISM-K2-03",
      "level": "K2",
      "requirement": "MUSS",
      "spec_refs": ["M-02", "PEP-2"],
      "test": "test_k2_governed.py::test_pii_masked_before_inference",
      "outcome": "passed",
      "duration_s": 0.412,
      "message": ""
    }
  ]
}
```

Die Zahlen im Beispiel sind **illustrativ**. Felder:

Feld	Bedeutung
levels.<K>.achieved	Stufe erreicht (alle MUSS-Tests dieser und der unteren Stufen bestanden)
must_failed / must_not_run	fehlgeschlagene bzw. übersprungene MUSS-Tests
should_failed	fehlgeschlagene SOLLTE-Tests
achieved_level	höchste erreichte Stufe oder none
results[].outcome	passed, failed, skipped

Manuelle Prüfungen werden im JSON nicht automatisch erfasst. Sie SOLLTEN in einem ergänzenden Prüfprotokoll mit denselben IDs dokumentiert werden; eine Stufe mit manuellen MUSS-Prüfungen (K2-14; K3-06) gilt erst mit diesem Protokoll als vollständig belegt.

6.2 Badge

Die Suite schreibt neben dem Bericht:

- aism-report-badge.json im **Endpoint-Format von shields.io** ({"schemaVersion": 1, "label": "AISM", "message": "K2 Governed", "color": "97ca00"}), einbindbar über <https://img.shields.io/endpoint?url=<öffentliche URL der JSON-Datei>>,
- aism-report-badge.svg als lokal erzeugtes Badge ohne externen Dienst.

Ein Badge DARF nur zusammen mit dem zugehörigen Bericht (Datum, Suite-Version, Policy-Version) veröffentlicht werden.

7. Aktueller Stand der Referenz (05.10.2026, Docker-Lauf)

7.1 Aufbau des Laufs

- Docker Engine 29.8.2 mit Compose v2 auf einem Debian-Testhost (ohne systemd, Storage-Treiber vfs), Firewall-Backend iptables-nft.
- Stack: docker-compose.yml + tests/compose.conformance.yml + ../deploy/firewall/test/compose.ipv6.yml (Dual-Stack). Gestartet: governance-proxy, orchestrator (beide lokal gebaut), qdrant, Capture-Mock aism-mock (statt llama.cpp, Embedding, n8n und SearXNG), Test-IdP mock-idp. Nicht gestartet: Open WebUI, llama.cpp (keine GPU), n8n, SearXNG, Perplexica.

- Referenz-Host-Firewall Variante A (`../deploy/firewall/docker-user.sh`) aktiv.
- Runner: Container `aism-runner` (Image aus `tests/Dockerfile`), nur im Netz `aism_frontend`, über `tests/run_in_docker.sh`. Benutzer-JWT (HS256, Open-WebUI-Format, `groups: [it-ops]`) und OIDC-Tokens des Test-IdP werden zur Laufzeit erzeugt.
- Policy: `tests/testdata/policy.conformance.yaml`, zur Laufzeit von `tests/prepare_signed_policy.sh` mit einem **frisch erzeugten Test-Schlüssel** signiert (Identität `aism-conformance-test@localhost`) und als Verzeichnis `read-only` gemountet. Der private Schlüssel liegt nur unter `tests/run/` (nicht im Paket).
- `AIMS_POLICY_FAULT_INJECTION=1` (K3-08), `AIMS_ALLOWED_SIGNERS` (K3-01).

7.2 Ergebnis

Zuerst lief das Szenario Cloud-Fallback (7.3), danach der vollständige Standardlauf auf demselben Audit-Volumen (K3-04 wertet die `egress.cloud`-Einträge des Szenarios aus).

Stufe	Tests	bestanden	fehlgeschlagen	übersprungen	erreicht
K1 Basic	10 (9 MUSS, 1 SOLLTE)	10	0	0	ja
K2 Governed	19 (13 MUSS, 6 SOLLTE)	19	0	0	ja (manuelle Prüfung K2-14 s. 7.4)
K3 Sovereign/Auditable	8 (8 MUSS)	8	0	0	ja (manuelle Prüfung K3-06 nicht anwendbar, s. 7.4)

Ergebnis: **achieved_level: K3**, 37 bestanden, 0 fehlgeschlagen, 0 übersprungen. Berichte und Badges: `reports/2026-10-05-docker/` (`aism-report.json`, `aism-report-cloud-fallback.json`). Der Lauf wurde nach der Umbenennung des Projekts in AISM (Header `X-AISM-*`, Variablen `AIMS_*`, Images `ghcr.io/marksen23/aism-*`, Policy-`apiVersion aism/v1alpha1`) am 05.10.2026 um 09:15 CEST wiederholt, mit demselben Ergebnis. Die Berichte im Verzeichnis stammen aus diesem Lauf.

Ein früherer Docker-Lauf **ohne** vorheriges Cloud-Szenario ergab 36 bestanden und K3-04 übersprungen (K2). Der K3-Nachweis braucht also beide Läufe.

7.3 Szenario Cloud-Fallback (AISM-K3-04, AISM-K3-10)

`tests/prepare_cloud_scenario.sh` signiert die Cloud-Test-Policy `tests/testdata/policy.conformance-cloud.yaml` und erzeugt eine Test-CA mit Serverzertifikat. Der CA-Schlüssel wird danach gelöscht. `tests/compose.cloud-fallback.yml` startet einen HTTPS-Mock-Provider im `egress`-Netz; der lokale Inferenz-Mock `llama-mock` wird **gestoppt**. Ergebnis (6 Tests, alle bestanden; `scenario_cloud_fallback.py`):

- Rolle `it-ops`, keine PII → Antwort vom Cloud-Modell (JSON und Stream), TLS-Prüfung gegen die Test-CA aktiv, keine `X-AISM-*`-Header beim Provider.
- Prompt mit E-Mail-Adresse → Regel `restricted-local-only` → 503 `upstream_unavailable`, **kein** Request beim Provider.
- Rolle ohne `it-ops` → `baseline-local` → 503, kein Request beim Provider.
- Audit `egress.cloud` mit `reason: local_unavailable`, `rule_ids`, `provider`, Policy-Digest und `payload_sha256`; Hash-Kette intakt.

7.4 Manuelle Prüfungen (informell, ohne formales Prüfprotokoll)

ID	Ergebnis	Beobachtung
AIMS-K2-14	erfüllt (Lauf ohne Docker, 05.10.2026)	NER-Modell in der Policy auf ein nicht vorhandenes Modell gesetzt: <code>/health</code> → 503, Chat-Request → 503 <code>pii_detector_unavailable</code> , keine Weiterleitung. Im Docker-Lauf nicht wiederholt.
AIMS-K2-17	erfüllt (Lauf ohne Docker)	Rolle <code>staff</code> erhält E-Mail im Klartext, IBAN als <code><IBAN_1></code> ; Rolle <code>it-ops</code> beide; <code><SECRET_1></code> bleibt für alle maskiert
AIMS-K3-06	nicht anwendbar	keine Modelldateien im Testaufbau (Mock statt <code>llama.cpp</code>)

7.5 Einordnung

- Der Lauf belegt das Verhalten der **Prototypen** mit **Mocks** statt Open WebUI, `llama.cpp`, `n8n` und `SearXNG`. Er ist kein Nachweis für eine produktive Installation.
- OIDC (K2-21) ist nur gegen den Test-IdP `tests/mock_idp.py` geprüft, nicht gegen einen echten IdP.
- K3-05 prüft bei lokal gebauten Images nur das Basis-Image; die gebauten Images sind reproduzierbar (README), aber nicht per Digest referenziert, solange sie nicht in einer Registry liegen.
- AIMS-K2-03 und -18 prüfen nur die synthetischen Testdaten. Eine erste Messung der Erkennungsquote auf einem synthetischen deutschen Datensatz steht in `pii-eval/README.md`; reale Daten sind nicht gemessen.
- Firewall-Ergebnisse auf dem Docker-Host (inkl. IPv6): `../deploy/firewall/README.md`.

7.6 Früherer Lauf ohne Docker (05.10.2026)

Gateway-Prototyp und Orchestrator-Stub lokal (Python 3.13, uvicorn), Runner in einem Netz-Namespace. Ergebnis **K2**: K1 10/10, K2 15/15, K3 3 bestanden, 2 fehlgeschlagen (K3-01 keine Signatur, K3-05 keine Digests), 1 übersprungen (K3-04). Berichte: [reports/2026-10-05-lokal-prototyp/](#). Die Pfade in diesen älteren Berichten wurden bei der Umbenennung mechanisch an den neuen Verzeichnisnamen angepasst; die Ergebnisse sind unverändert.

8. Offene Punkte

1. AISM-K2-14 (Ausfall des PII-Detektors) als automatisierte Fehlerinjektion; K3-06 (Modell-Prüfsummen) mit echten Modelldateien.
2. Lauf mit den echten Komponenten (Open WebUI, llama.cpp auf GPU, n8n, SearXNG) und einem echten IdP; Lauf in CI.
3. K3-04 hängt davon ab, dass vorher Cloud-Egress stattfand. Für Installationen mit dauerhaft deaktiviertem Cloud-Routing ist zu klären, ob K3-04 „nicht anwendbar“ statt „übersprungen“ werten soll.
4. Prüfung des Bestätigungsablaufs über die Oberfläche (Open WebUI) fehlt; die Suite prüft nur die API.